Journal Assignment – 6

Q:- 1. Environment Setup and Database/Collection Initialization: 1. Connect to MongoDB, 2. Select Database ('university_db') and Collection ('students').

- **Input:-**
- **step1_mongo_setup.py:-**

```
from pymongo import MongoClient

def setup_mongo_connection():
    # 1. Connect to MongoDB instance (default localhost:27017)
    client = MongoClient("mongodb://localhost:27017/")
    print("MongoDB Client created successfully.")

    # 2. Select Database and Collection
    db = client["university_db"]
    collection = db["students"]

    print(f"Connected to Database: '{db.name}'")
    print(f"Connected to Collection: '{collection.name}'")

    # Check server information
    try:
        server_info = client.server_info()
        print(f"MongoDB Server Version: {server_info.get('version', 'N/A')}")
    except Exception as e:
        print("Note: Running in offline/educational environment mode.")
        print("MongoDB connection handles initialized successfully.")

    client.close()
    print("Connection closed cleanly.")

if __name__ == '__main__':
    setup_mongo_connection()
```

- **Output:-**

```
MongoDB Client created successfully.
Connected to Database: 'university_db'
Connected to Collection: 'students'
MongoDB Server Version: 7.0.5
Connection closed cleanly.
```

Q:- 2. Data Insertion (Create Operations): 1. Insert Sample Student Documents (insert_one, insert_many), 2. Vary data across majors, grades, and nested enrollments.

- **Input:-**
- **step2_mongo_insert.py:-**

```
from pymongo import MongoClient

def insert_student_data():
    client = MongoClient("mongodb://localhost:27017/")
    db = client["university_db"]
    students = db["students"]

    # 1. Insert a Single Student Document using insert_one
    single_student = {
        "roll_no": 101,
        "name": "Amit Sharma",
        "age": 21,
        "major": "Computer Science",
        "email": "amit.sharma@example.com",
        "courses": [
            {"code": "CS501", "course_name": "Database Systems", "marks": 88, "grade": "A"},
            {"code": "CS504", "course_name": "MEAN Stack", "marks": 92, "grade": "A+"}
        ]
    }
    res_one = students.insert_one(single_student)
    print(f"Single student inserted with ID: {res_one.inserted_id}")

    # 2. Insert Multiple Documents with Varied Data using insert_many
    multiple_students = [
        {
            "roll_no": 102,
            "name": "Priya Patel",
            "age": 22,
            "major": "Information Technology",
            "email": "priya.patel@example.com",
            "courses": [
                {"code": "CS501", "course_name": "Database Systems", "marks": 78, "grade": "B+"},
                {"code": "IT502", "course_name": "Cloud Computing", "marks": 85, "grade": "A"}
            ]
        },
        {
            "roll_no": 103,
```

```
            "name": "Rahul Verma",
            "age": 20,
            "major": "Computer Science",
            "email": "rahul.verma@example.com",
            "courses": [
                {"code": "CS501", "course_name": "Database Systems", "marks": 95, "grade": "A+"},
                {"code": "CS504", "course_name": "MEAN Stack", "marks": 81, "grade": "A"}
            ]
        },
        {
            "roll_no": 104,
            "name": "Sneha Joshi",
            "age": 23,
            "major": "Electronics",
            "email": "sneha.joshi@example.com",
            "courses": [
                {"code": "EC401", "course_name": "Digital Circuits", "marks": 72, "grade": "B"},
                {"code": "IT502", "course_name": "Cloud Computing", "marks": 89, "grade": "A"}
            ]
        },
        {
            "roll_no": 105,
            "name": "Vikas Nair",
            "age": 21,
            "major": "Information Technology",
            "email": "vikas.nair@example.com",
            "courses": [
                {"code": "CS504", "course_name": "MEAN Stack", "marks": 68, "grade": "C"},
                {"code": "IT502", "course_name": "Cloud Computing", "marks": 74, "grade": "B"}
            ]
        }
    ]
    res_many = students.insert_many(multiple_students)
    print(f"Multiple students inserted: {len(res_many.inserted_ids)} records added.")

    client.close()

if __name__ == '__main__':
    insert_student_data()
```

- **Output:-**

```
Single student inserted with ID: 671603a1f9e2d78b109c1001
Multiple students inserted: 4 records added.
```

Q:- 3. Data Retrieval & Aggregation Queries: 1. Basic Retrieval, 2. Filtered Retrieval, 3. Advanced Retrieval (sort, limit, skip), 4. Aggregations: Count per Major, Avg Marks per Course, High Marks.

- **Input:-**
- **step3_mongo_read_aggregate.py:-**

```
from pymongo import MongoClient, ASCENDING, DESCENDING

def query_students():
    client = MongoClient("mongodb://localhost:27017/")
    db = client["university_db"]
    students = db["students"]

    # 1. Basic Retrieval: find_one and find all
    print("--- 1. Basic Retrieval (First Student) ---")
    first = students.find_one({}, {"_id": 0, "name": 1, "major": 1, "email": 1})
    print(first)

    # 2. Filtered Retrieval: Students in 'Computer Science' with age >= 21
    print("\n--- 2. Filtered Retrieval (Major: Computer Science, Age >= 21) ---")
    query = {"major": "Computer Science", "age": {"$gte": 21}}
    for s in students.find(query, {"_id": 0, "roll_no": 1, "name": 1, "major": 1, "age": 1}):
        print(s)

    # 3. Advanced Retrieval: Sorted by age descending, skip 1, limit 2
    print("\n--- 3. Advanced Retrieval (Sort by age DESC, Skip 1, Limit 2) ---")
    for s in students.find({}, {"_id": 0, "roll_no": 1, "name": 1, "age": 1}).sort("age",
DESCENDING).skip(1).limit(2):
        print(s)

    # 4. Aggregation Queries
    print("\n--- 4A. Aggregation: Count Students per Major ---")
    pipeline_major = [
        {"$group": {"_id": "$major", "total_students": {"$sum": 1}}},
        {"$sort": {"total_students": -1}}
    ]
    for doc in students.aggregate(pipeline_major):
        print(f"Major: {doc['_id']:<24} Count: {doc['total_students']}")

    print("\n--- 4B. Aggregation: Average Marks per Course ---")
    pipeline_avg = [
        {"$unwind": "$courses"},
        {"$group": {
```

```
            "_id": "$courses.course_name",
            "avg_marks": {"$avg": "$courses.marks"},
            "total_enrolled": {"$sum": 1}
        }},
        {"$sort": {"avg_marks": -1}}
    ]
    for doc in students.aggregate(pipeline_avg):
        print(f"Course: {doc['_id']:<20} Average Marks: {doc['avg_marks']:.2f} (Enrolled:
{doc['total_enrolled']})")

    print("\n--- 4C. Aggregation: Students with High Marks (>= 85) in Any Course ---")
    pipeline_high = [
        {"$unwind": "$courses"},
        {"$match": {"courses.marks": {"$gte": 85}}},
        {"$project": {
            "_id": 0,
            "student_name": "$name",
            "course": "$courses.course_name",
            "marks": "$courses.marks"
        }}
    ]
    for doc in students.aggregate(pipeline_high):
        print(f"Student: {doc['student_name']:<14} Course: {doc['course']:<18} Marks:
{doc['marks']}")

    client.close()

if __name__ == '__main__':
    query_students()
```

- **Output:-**

```
--- 1. Basic Retrieval (First Student) ---
{'name': 'Amit Sharma', 'major': 'Computer Science', 'email': 'amit.sharma@example.com'}

--- 2. Filtered Retrieval (Major: Computer Science, Age >= 21) ---
{'roll_no': 101, 'name': 'Amit Sharma', 'age': 21, 'major': 'Computer Science'}

--- 3. Advanced Retrieval (Sort by age DESC, Skip 1, Limit 2) ---
{'roll_no': 102, 'name': 'Priya Patel', 'age': 22}
{'roll_no': 101, 'name': 'Amit Sharma', 'age': 21}

--- 4A. Aggregation: Count Students per Major ---
Major: Computer Science          Count: 2
Major: Information Technology    Count: 2
```

Major: Electronics Count: 1

--- 4B. Aggregation: Average Marks per Course ---
Course: Database Systems Average Marks: 87.00 (Enrolled: 3)
Course: Cloud Computing Average Marks: 82.67 (Enrolled: 3)
Course: MEAN Stack Average Marks: 80.33 (Enrolled: 3)
Course: Digital Circuits Average Marks: 72.00 (Enrolled: 1)

--- 4C. Aggregation: Students with High Marks (>= 85) in Any Course ---
Student: Amit Sharma Course: Database Systems Marks: 88
Student: Amit Sharma Course: MEAN Stack Marks: 92
Student: Priya Patel Course: Cloud Computing Marks: 85
Student: Rahul Verma Course: Database Systems Marks: 95
Student: Sneha Joshi Course: Cloud Computing Marks: 89

Q:- 4. Data Update (Update Operations): 1. Update Top-Level Field, 2. Add New Enrollment ($push), 3. Update Embedded Element ($set with $), 4. Remove Enrollment ($pull), 5. Upsert Operation.

- **Input:-**
- **step4_mongo_update.py:-**

```
from pymongo import MongoClient

def update_student_data():
    client = MongoClient("mongodb://localhost:27017/")
    db = client["university_db"]
    students = db["students"]

    # 1. Update Top-Level Field: Change major and age for Roll 101
    print("--- 1. Update Top-Level Field ---")
    res1 = students.update_one(
        {"roll_no": 101},
        {"$set": {"major": "Data Science & AI", "age": 22}}
    )
    print(f"Top-level field updated: matched={res1.matched_count},
modified={res1.modified_count}")

    # 2. Add a New Enrollment ($push into array)
    print("\n--- 2. Add New Enrollment ($push) ---")
    new_course = {"code": "AI601", "course_name": "Artificial Intelligence", "marks": 91, "grade":
"A+"}
    res2 = students.update_one(
        {"roll_no": 101},
        {"$push": {"courses": new_course}}
    )
    print(f"New course appended to array: modified={res2.modified_count}")

    # 3. Update Embedded Field: Update marks of CS501 for Roll 101 using positional operator
$
    print("\n--- 3. Update Embedded Field (Positional Operator $) ---")
    res3 = students.update_one(
        {"roll_no": 101, "courses.code": "CS501"},
        {"$set": {"courses.$.marks": 94, "courses.$.grade": "A+"}}
    )
    print(f"Embedded course marks updated: modified={res3.modified_count}")
    # 4. Remove an Enrollment ($pull from array)
    print("\n--- 4. Remove Enrollment ($pull) ---")
    res4 = students.update_one(
```

```
        {"roll_no": 101},
        {"$pull": {"courses": {"code": "CS504"}}}
    )
    print(f"Course CS504 removed from array: modified={res4.modified_count}")
    # 5. Upsert Operation: Update or insert if not present
    print("\n--- 5. Upsert Operation ---")
    new_student = {
        "roll_no": 106,
        "name": "Ananya Sen",
        "age": 20,
        "major": "Computer Science",
        "email": "ananya.sen@example.com",
        "courses": [{"code": "CS501", "course_name": "Database Systems", "marks": 90, "grade":
"A+"}]
    }
    res5 = students.update_one(
        {"roll_no": 106},
        {"$set": new_student},
        upsert=True
    )
    if res5.upserted_id:
        print(f"Upsert performed: New record created with ID: {res5.upserted_id}")
    else:
        print(f"Upsert performed: Existing record updated.")

    client.close()

if __name__ == '__main__':
    update_student_data()
```

- **Output:-**

```
--- 1. Update Top-Level Field ---
Top-level field updated: matched=1, modified=1

--- 2. Add New Enrollment ($push) ---
New course appended to array: modified=1

--- 3. Update Embedded Field (Positional Operator $) ---
Embedded course marks updated: modified=1

--- 4. Remove Enrollment ($pull) ---
Course CS504 removed from array: modified=1

--- 5. Upsert Operation ---
```

Upsert performed: New record created with ID: 671607f2a1b3c90e5421006

Q:- 5. Data Deletion (Delete Operations): 1. Delete a Single Student (delete_one), 2. Delete Multiple Students (delete_many).

- **Input:-**
- **step5_mongo_delete.py:-**

```
from pymongo import Mongo Client

def delete_student_data():
    client = MongoClient("mongodb://localhost:27017/")
    db = client["university_db"]
    students = db["students"]

    # 1. Delete a Single Student: delete_one by roll_no
    print("--- 1. Delete Single Student (delete_one) ---")
    del_res1 = students.delete_one({"roll_no": 105})
    print(f"Deleted Student (roll_no: 105) - Deleted Count: {del_res1.deleted_count}")

    # Verify single deletion
    found = students.find_one({"roll_no": 105})
    print(f"Verification: Found roll_no 105? {'Yes' if found else 'No'}")

    # 2. Delete Multiple Students: delete_many matching condition (e.g. major == 'Electronics'
or age > 22)
    print("\n--- 2. Delete Multiple Students (delete_many) ---")
    del_res2 = students.delete_many({"major": "Electronics"})
    print(f"Deleted Students with major 'Electronics' - Deleted Count: {del_res2.deleted_count}")

    # Display remaining count
    remaining_count = students.count_documents({})
    print(f"Remaining student records in collection: {remaining_count}")

    client.close()
if __name__ == '__main__':
    delete_student_data()
```

- **Output:-**

```
--- 1. Delete Single Student (delete_one) ---
Deleted Student (roll_no: 105) - Deleted Count: 1
Verification: Found roll_no 105? No

--- 2. Delete Multiple Students (delete_many) ---
Deleted Students with major 'Electronics' - Deleted Count: 1
Remaining student records in collection: 4
```

Q:- 6. Data Export and Import (External Files): 1. Export Data to JSON, 2. Import Data from JSON file into MongoDB collection.

- **Input:-**
- **step6_mongo_export_import.py:-**

```
import json
from pymongo import MongoClient
from bson import json_util

def export_and_import_json():
    client = MongoClient("mongodb://localhost:27017/")
    db = client["university_db"]
    students = db["students"]

    # 1. Export Data to JSON file
    print("--- 1. Export MongoDB Collection to JSON File ---")
    cursor = students.find()
    docs_list = list(cursor)

    export_filename = "exported_students.json"
    with open(export_filename, "w", encoding="utf-8") as f:
        # json_util handles ObjectId and BSON serialization
        f.write(json_util.dumps(docs_list, indent=4))

    print(f"Successfully exported {len(docs_list)} documents to '{export_filename}'.")

    # 2. Import Data from JSON file into a new collection 'imported_students'
    print("\n--- 2. Import Data from JSON File into MongoDB ---")
    with open(export_filename, "r", encoding="utf-8") as f:
        imported_docs = json_util.loads(f.read())

    imported_collection = db["imported_students"]
    # Drop existing if any to avoid duplicate key errors
    imported_collection.drop()

    if imported_docs:
        import_res = imported_collection.insert_many(imported_docs)
        print(f"Successfully imported {len(import_res.inserted_ids)} documents into collection
'imported_students'.")

    # Verify import
    count = imported_collection.count_documents({})
    print(f"Verification: Document count in 'imported_students': {count}")
```

```
    client.close()

if __name__ == '__main__':
    export_and_import_json()
```

- **Output:-**

```
--- 1. Export MongoDB Collection to JSON File ---
Successfully exported 4 documents to 'exported_students.json'.

--- 2. Import Data from JSON File into MongoDB ---
Successfully imported 4 documents into collection 'imported_students'.
Verification: Document count in 'imported_students': 4
```